OS202 – Systemes paralleles et distribues

Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

MENESES GAMBOA Carlos

VASQUEZ TORRES Jair

Ecole Nationale des techniques avancees

Mars 2026

Table des matières

1 Introduction et environnement d'execution **2**
1.1 Methodologie experimentale et reproductibilite . . . 2
1.2 Machine de test . . . 2
1.3 Organisation du depot . . . 2

2 Q0 : Analyse de l'existant et baseline monocoeur **3**
2.1 Pipeline applicatif . . . 3
2.2 Hotspots theoriques . . . 3
2.2.1 Avance des fourmis . . . 3
2.2.2 Traitement global de la grille . . . 4
2.3 Baseline sequentielle . . . 4

3 Q1 : Instrumentation et definition du scope **4**
3.1 Instrumentation par `std::chrono` . . . 4
3.2 Sanity checks et scope de mesure . . . 4
3.3 Tableau de resultats Q1 . . . 5

4 Q2 : Vectorisation (AoS vs SoA) et optimisations **5**
4.1 Theorie : passage de AoS a SoA . . . 5
4.2 Le paradoxe des performances : pourquoi SoA n'a pas apporte un gain magique . 6
4.3 Les vraies optimisations du code final . . . 6
4.3.1 Preallocation des structures auxiliaires . . . 6
4.3.2 Separation des fourmis selon l'etat . . . 6
4.3.3 Reduction du cout du generateur pseudo-aleatoire . . . 7
4.3.4 Autres ameliorations structurelles . . . 7
4.4 Tableau comparatif Q1 vs Q2 . . . 7
4.5 Reproductibilite et validation fonctionnelle . . . 7
4.6 Conclusion Q2 . . . 8

1 Introduction et environnement d'execution

TODO : rediger une introduction generale qui presente le projet, les objectifs, le contexte de l'optimisation et les grandes lignes du rapport.

1.1 Methodologie experimentale et reproductibilite

> Le protocole vise une comparaison reproductible. La graine initiale est fixe a `2026` dans les executables Qx. Un *warm-up* complet est execute avant les repetitions mesurees afin de chauffer les caches et n'est jamais comptabilise dans les resultats exportes. La validation fonctionnelle repose sur deux indicateurs : `Food_KPI`, qui compte la nourriture ramenee au nid, et un `Checksum` construit a partir de l'etat final des fourmis et des pheromones. Toute optimisation retenue doit conserver ces deux quantites.

Le protocole cible du projet est un **scope compute strict**, c'est-a-dire des mesures concentrees sur la boucle de calcul avec rendu SDL desactive via `-gui 0`. Cependant, les fichiers archives actuellement presents dans `results/` contiennent un `T_render` non nul, ce qui signifie que la campagne archivée ne correspond pas a un headless strict.

Par rigueur scientifique, le present rapport reproduit donc **les valeurs effectivement archivees** dans le depot. La campagne instrumentee Q1/Q2 disponible dans `results/` repose, elle, sur **5 repetitions** avec warm-up prealable.

1.2 Machine de test

Le fichier `results/machine_info.txt` n'est pas present dans l'archive courante. Les informations ci-dessous ont donc ete reconstituees a partir de l'environnement d'execution local et des metadonnees deja presentes dans le depot.

Table 1 – Environnement d'execution utilise pour les mesures et l'analyse.

Champ	Valeur
Systeme	Linux 6.6.87.2-microsoft-standard-WSL2 x86_64
Processeur	11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Coeurs physiques	4
Coeurs logiques	8
Threads par coeur	2
Cache L1d / L1i	192 KiB / 128 KiB
Cache L2	5 MiB
Cache L3	8 MiB
Memoire vive	7.6 GiB
Compilateur C++	g++ 11.4.0

1.3 Organisation du depot

Le depot est structure par etapes independantes :

— `Q0/` : version de reference monocoeur ;

— `Q1/` : version instrumentee avec decomposition temporelle ;
— `Q2/` : version SoA et optimisations du noyau chaud ;
— `results/` : export des mesures numeriques utilisees dans ce rapport.

Cette organisation garantit la tracabilite des modifications et facilite la comparaison entre versions.

2 Q0 : Analyse de l'existant et baseline monocoeur

2.1 Pipeline applicatif

Le pipeline global de l'application peut etre resume comme suit :

Generation du terrain fractal
$\Downarrow$
Initialisation des pheromones et des fourmis
$\Downarrow$
Boucle de simulation
deplacement des fourmis $\rightarrow$ evaporation $\rightarrow$ mise a jour des pheromones $\rightarrow$ rendu / sorties
$\Downarrow$
Export des temps, du `Food_KPI` et du `Checksum`

Cette decomposition met deja en evidence trois blocs de calcul dominants :

— le deplacement des fourmis, avec une complexite proportionnelle au nombre de fourmis et a leur budget de mouvement ;
— l'evaporation globale, qui parcourt la grille complete ;
— la mise a jour des pheromones, elle aussi tres dependante de la taille de la grille.

2.2 Hotspots theoriques

Avant toute instrumentation, deux goulots d'etranglement etaient deja identifies.

2.2.1 Avance des fourmis

Le deplacement des fourmis constitue le principal candidat au statut de point chaud theorique. Chaque iteration parcourt toutes les fourmis, lit les pheromones voisines, applique une decision exploration / exploitation, met a jour l'etat de charge et peut effectuer plusieurs micro-deplacements avant d'epuiser le budget de temps. Cette phase est a la fois :

— intensive en branchements ;
— sensible a la localite memoire ;
— dependante du generateur pseudo-aleatoire.

2.2.2 Traitement global de la grille

L'evaporation et la mise a jour des pheromones parcourent la grille entiere a chaque iteration. Leur cout est donc structurellement lie a la taille du paysage fractal. Ce sont des noyaux plus reguliers que le deplacement des fourmis, mais potentiellement couteux en trafic memoire.

2.3 Baseline sequentielle

Le fichier `results/Q0_baseline.csv` contient la baseline monocoeur archivee. Il fournit un temps sequentiel de reference T_s ainsi que les indicateurs de validation.

TABLE 2 – Baseline sequentielle Q0 extraite de `results/Q0_baseline.csv`.

Metrique	Valeur
Nombre de repetitions archivees	2
Temps moyen T_s	33 788.16 ms
Ecart-type	2804.10 ms
Premiere iteration avec nourriture	1236
Food_KPI	296
Checksum	1641247467920585528

Cette baseline sert de point de reference algorithmique. Elle montre deja que la simulation complete se situe dans un regime de calcul non trivial et justifie pleinement une instrumentation fine dans l'etape Q1.

3 Q1 : Instrumentation et definition du scope

3.1 Instrumentation par `std::chrono`

L'etape Q1 introduit une instrumentation par sous-etapes a l'aide de `std::chrono`. Le temps total `T_total` encadre la boucle temporelle principale, tandis que quatre sous-mesures sont isolees :

- `T_move_ants` : deplacement des fourmis ;
- `T_evap` : evaporation globale des pheromones ;
- `T_pher_update` : application de la nouvelle carte de pheromones ;
- `T_render` : rendu SDL.

Ce decoupage permet de distinguer :

- les noyaux calculatoires internes ;
- le cout de l'interface graphique ;
- la coherence globale entre la somme des sous-etapes et le temps total.

3.2 Sanity checks et scope de mesure

Le code affiche un *sanity check* pour verifier que :

$$T_{\text{total}} \approx T_{\text{move_ants}} + T_{\text{evap}} + T_{\text{pher_update}} + T_{\text{render}}.$$

En mode headless, un second controle impose $T_{\text{render}} \approx 0$. Ce point correspond au scope experimental vise par le projet : isoler strictement le calcul.

Cependant, les CSV archives dans `results/` pour Q1 contiennent un `T_render` non nul. Les valeurs reportees ci-dessous reproduisent donc fidelement la campagne disponible, et non une campagne recalculée en `-gui 0`.

3.3 Tableau de resultats Q1

TABLE 3 – Decomposition temporelle Q1 extraite de `results/Q1_timings_breakdown.csv`.

Sous-etape	Moyenne (ms)	Ecart-type (ms)	Part dans T_{total}
$T_{\text{move_ants}}$	15253.78	1898.42	18.13 %
T_{evap}	1664.29	48.18	1.98 %
$T_{\text{pher_update}}$	38.92	3.79	0.05 %
T_{render}	67094.22	1198.94	79.76 %
T_{total}	84118.09	2178.69	100.00 %

Les indicateurs fonctionnels associes a cette campagne sont :

- `First_Iteration = 1236`
- `Food_KPI = 4353`
- `Checksum = 1479050492479337403`

La conclusion operationnelle de Q1 est donc claire : pour accelerer la simulation, il faut prioritairement attaquer le noyau `move_ants`, puis seulement dans un second temps les traitements plus reguliers comme l'evaporation.

4 Q2 : Vectorisation (AoS vs SoA) et optimisations

4.1 Theorie : passage de AoS a SoA

L'objectif theorique de Q2 etait de transformer le stockage des fourmis depuis une organisation de type *Array of Structures* (AoS) vers une organisation *Structure of Arrays* (SoA). Dans la version de reference, chaque fourmi etait un objet 'ant' contenant simultanement :

- sa graine pseudo-aleatoire ;
- son etat charge / non charge ;
- sa position courante.

En Q2, ces informations sont stockees dans des tableaux separes :

- `m_x` ;
- `m_y` ;
- `m_is_loaded` ;
- `m_seed`.

L'interet attendu d'une telle transformation est classique en calcul haute performance :

— meilleure localite spatiale ;
— meilleure exploitation des lignes de cache ;
— acces memoire plus reguliers ;
— fondation plus favorable a une vectorisation automatique ulterieure.

4.2 Le paradoxe des performances : pourquoi SoA n'a pas apporte un gain magique

L'analyse du code et des premiers essais a montre un point important : la version AoS initiale etait deja relativement compacte et bien adaptee au cache. Les objets `ant` restaient de petite taille et etaient stockes de maniere contigue dans un `std::vector<ant>`. Le passage a SoA n'attaquait donc pas un probleme memoire catastrophique.

En parallele, le noyau `advance_one` reste difficile a vectoriser automatiquement car il contient :

— des branchements dependant de l'etat de la fourmi ;
— des choix exploration / exploitation ;
— un `do/while` dans la branche d'exploration ;
— des trajectoires qui dependent des donnees et des tirages pseudo-aleatoires.

Le rapport du compilateur, synthese dans `Q2/results/reviews/revue_vectorisation_ant.md`, confirme cette lecture. Aucun des signaux utiles n'est marque comme `optimized` ; au contraire, le compilateur remonte des causes telles que `couldn't vectorize loop` ou `number of iterations cannot be computed`. En pratique, SoA etait une condition necessaire pour assainir le stockage, mais certainement pas une condition suffisante pour obtenir une vectorisation SIMD complete.

4.3 Les vraies optimisations du code final

La baisse du temps d'execution observee en Q2 provient en realite de plusieurs optimisations concretes du point chaud `move_ants`.

4.3.1 Preallocation des structures auxiliaires

La methode `resize` prealloue les tampons de travail :

— `consumed_time` ;
— `active_loaded_ids` ;
— `active_unloaded_ids` ;
— `next_loaded_ids` ;
— `next_unloaded_ids`.

Ce choix evite des allocations dynamiques dans la boucle chaude et stabilise le cout de l'ordonnancement par rondes.

4.3.2 Separation des fourmis selon l'etat

Les fourmis actives sont traitees en deux groupes, `loaded` et `unloaded`. Cette separation supprime un branchement recurrent pour choisir le canal de pheromone et simplifie le flot de controle.

4.3.3 Reduction du cout du generateur pseudo-aleatoire

Le gain majeur vient de la specialisation des tirages aleatoires.

En Q1, le noyau chaud utilisait des helpers generiques :

```
rand_double(0., 1., seed)
rand_int32(1, 4, seed)
```

En Q2, ces appels sont remplaces par :

```
next_seed(seed)
rand_choice_01(seed)
rand_dir_14(seed)
```

Cette modification reduit le cout CPU pour plusieurs raisons :

— suppression de l'appel a `std::fmod` dans le noyau chaud ;
— specialisation exacte des deux usages reels du RNG ;
— reduction des calculs generiques lies aux bornes `min_val` / `max_val` ;
— meilleur potentiel d'integration directe par le compilateur.

Autrement dit, l'acceleration finale ne vient pas d'une vectorisation automatique complete, mais d'un allegement massif du calcul scalaire repete a tres haute frequence.

4.3.4 Autres ameliorations structurelles

Le code final retient egalement :

— le passage de `std::vector<bool>` a `std::vector<std::uint8_t>` pour l'etat des fourmis ;
— le passage de `std::size_t` a `std::uint32_t` pour les graines ;
— l'appel direct a `mark_pheronome_xy` pour simplifier legerement la mise a jour des pheromones.

4.4 Tableau comparatif Q1 vs Q2

TABLE 4 – Comparaison des moyennes Q1 / Q2 a partir des fichiers de `results/`.

Metrique	Q1 (ms)	Q2 (ms)	Gain relatif	Facteur d'acceleration
$T_{\text{move_ants}}$	15253.78	6684.21	-56.18 %	2.282x
T_{evap}	1664.29	1532.42	-7.92 %	1.086x
$T_{\text{pher_update}}$	38.92	38.40	-1.35 %	1.014x
T_{render}	67094.22	64683.76	-3.59 %	1.037x
T_{total}	84118.09	73000.54	-13.22 %	1.152x

4.5 Reproductibilite et validation fonctionnelle

Les indicateurs de validation restent rigoureusement identiques entre Q1 et Q2 :

— `First_Iteration = 1236`

— `Food_KPI = 4353`
— `Checksum = 1479050492479337403`

Cette identite confirme que les optimisations introduites en Q2 ne modifient pas la logique sequentielle de reference. Le gain observe correspond bien a une acceleration d'execution, et non a une derivation du calcul.

4.6 Conclusion Q2

La Q2 montre une lecon importante en optimisation haute performance :

— le passage AoS -> SoA, pris isolenment, n'apporte pas necessairement un gain spectaculaire ;
— le rapport du compilateur est indispensable pour distinguer ce qui est vectorisable de ce qui ne l'est pas ;
— dans ce projet, les gains decisifs proviennent surtout de la simplification du noyau chaud et de la specialisation du generateur pseudo-aleatoire.

Le resultat est toutefois net :

— le temps de deplacement des fourmis est reduit d'un facteur 2.282× ;
— le temps total est reduit d'un facteur 1.152× ;
— la correction fonctionnelle est preservee.

Q2 constitue donc une etape techniquement reussie non pas parce qu'elle aurait force une vectorisation SIMD parfaite, mais parce qu'elle a permis de transformer un diagnostic de performance en optimisations concretes, mesurables et correctes.